LSTM

Recurrent Neural Networks (RNN)

It’s a type of neural network used to process sequential data.

What is Sequential Data?

Sequential data is data where the order of the elements matters.

Each data point has a relationship with the previous and the next points.

Common Examples

- Text:
Example: "I am going to the market"

If you shuffle the words like "market the to going am I" — it loses meaning.

- Time-series data:
Stock prices, weather temperatures, daily sales — the values depend on previous time steps.

- Audio:
Sounds and speech occur in a continuous flow — the sequence matters.

- Sensor data:
For example, heart rate from fitness trackers is logged over time — the order tells you the trend.

RNN Example

Imagine:

Input: ["I", "love", "mangoes"]

- Time Step 1: Input = "I", Output = h1
- Time Step 2: Input = "love", + h1 → Output = h2
- Time Step 3: Input = "mangoes", + h2 → Output = h3

RNN Features

Feature	RNN
Memory	Short
Handles long-term context?	No
Structure	Simple
Use today?	Rare

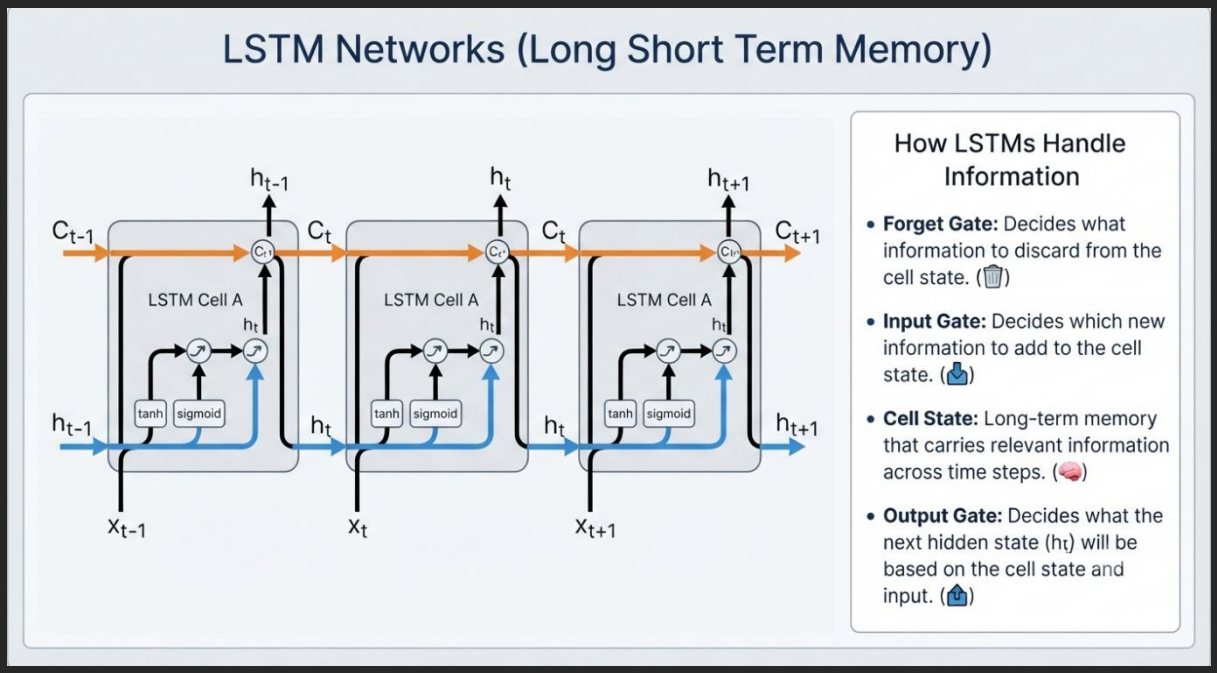
Limitations of Basic RNN

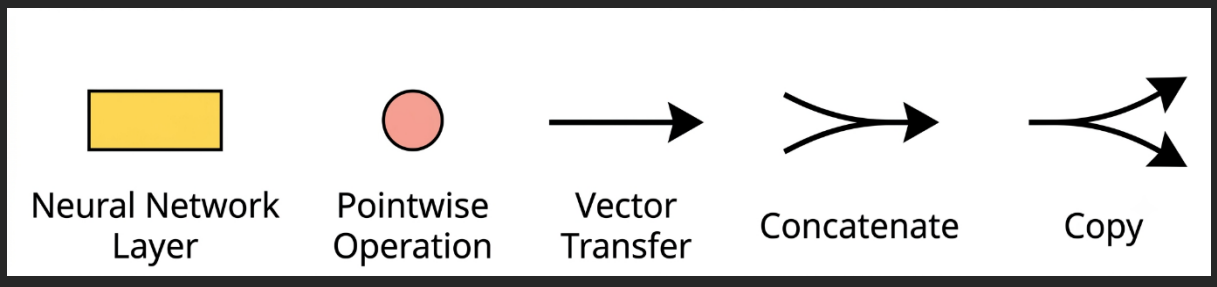
Problem	Why it Happens
Forgets long-term info (Vanishing gradients)	Because it passes memory step-by-step during backpropagation over long sequences
Slow training	No parallelization (it's sequential)

LSTM Networks (Long Short Term Memory)

LSTM is an advanced version of RNN (Recurrent Neural Network), but smarter — because it decides:

- What to remember
- What to forget
- What to output





Why was LSTM created?

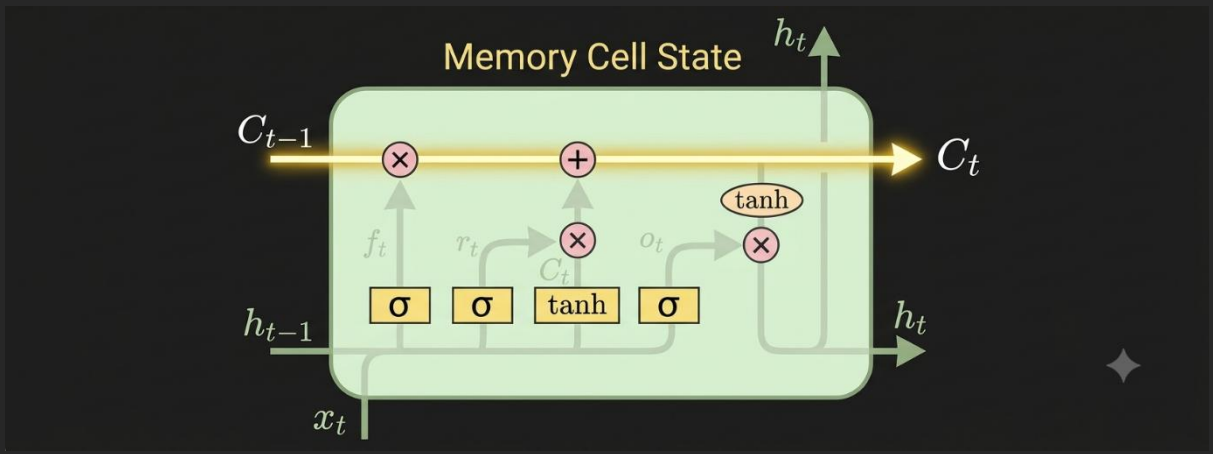
RNNs forget things quickly due to a problem called vanishing gradients.

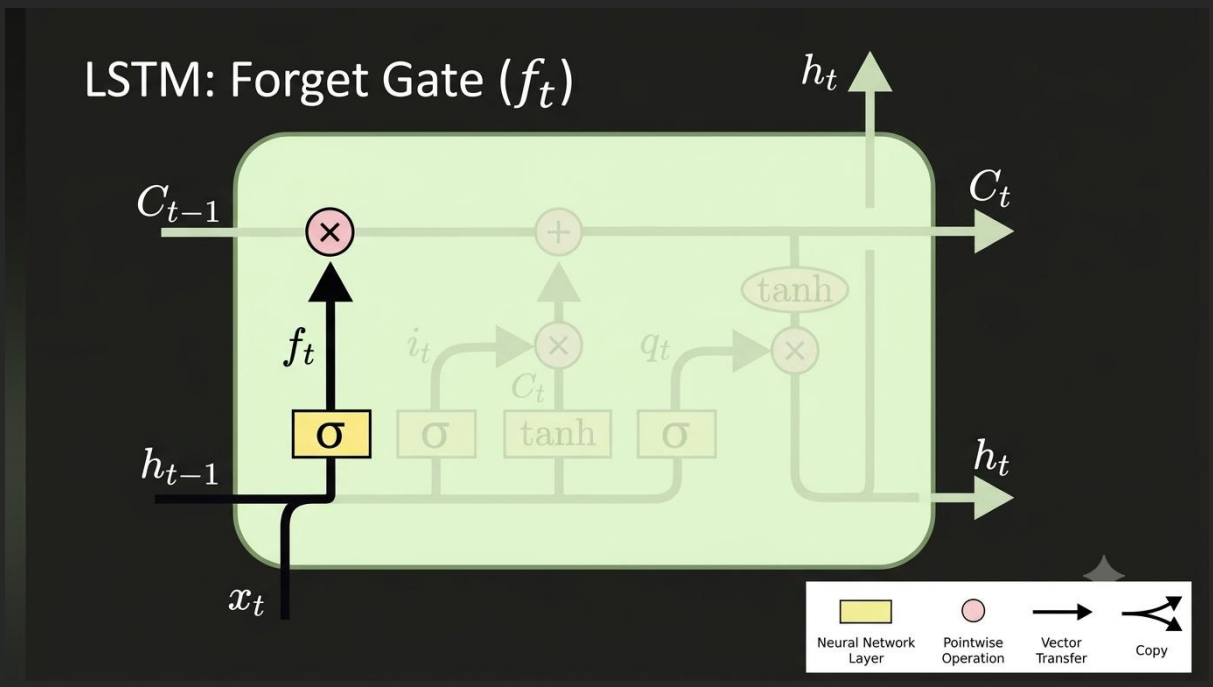
So if your model needs to remember something that happened many steps ago (like an important word in a long sentence), RNN fails, but LSTM handles it beautifully.

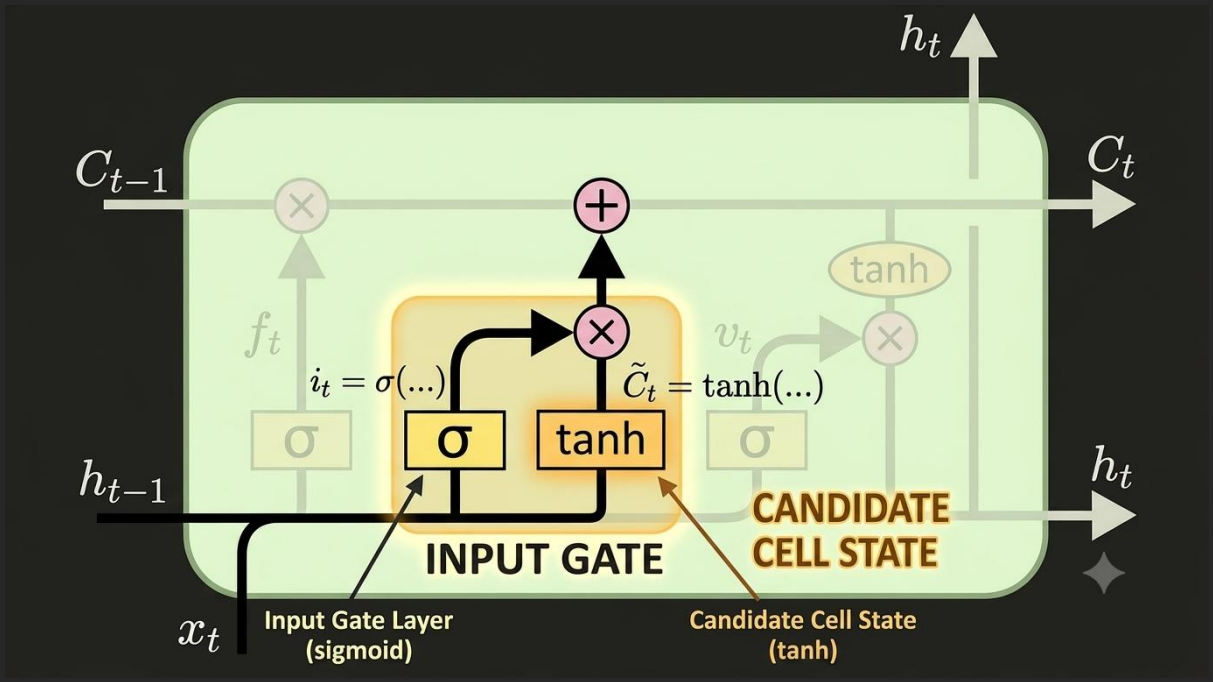
Component	Role
Cell State	Long-term memory highway (stores info)
Forget Gate	Decides what info to forget
Input Gate	Decides what new info to add
Output Gate	Decides what info to send to the next layer

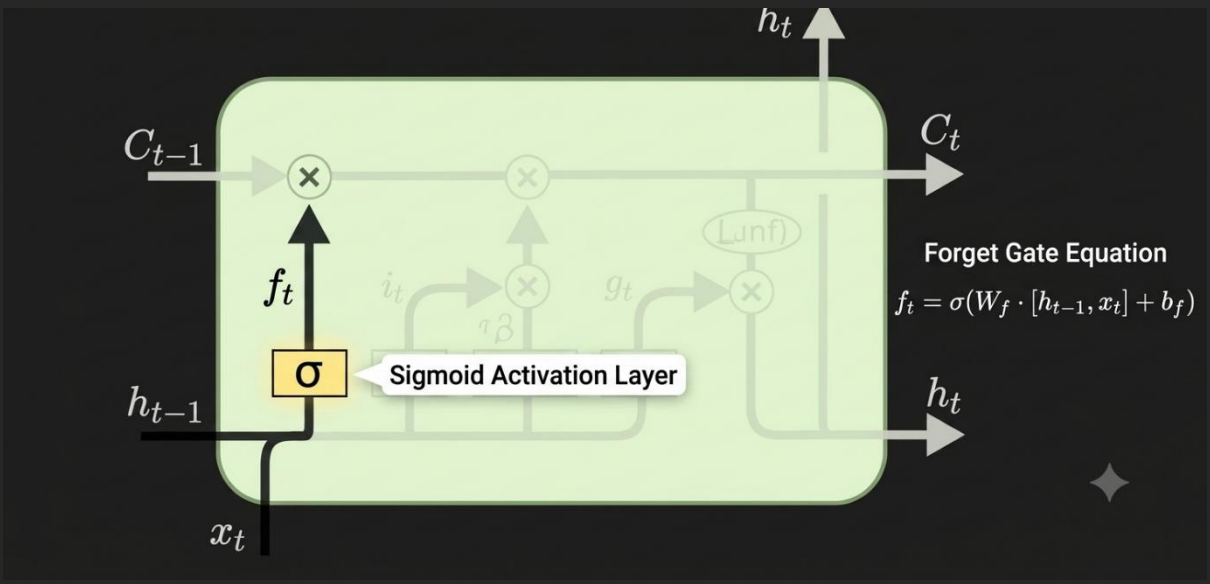
Example

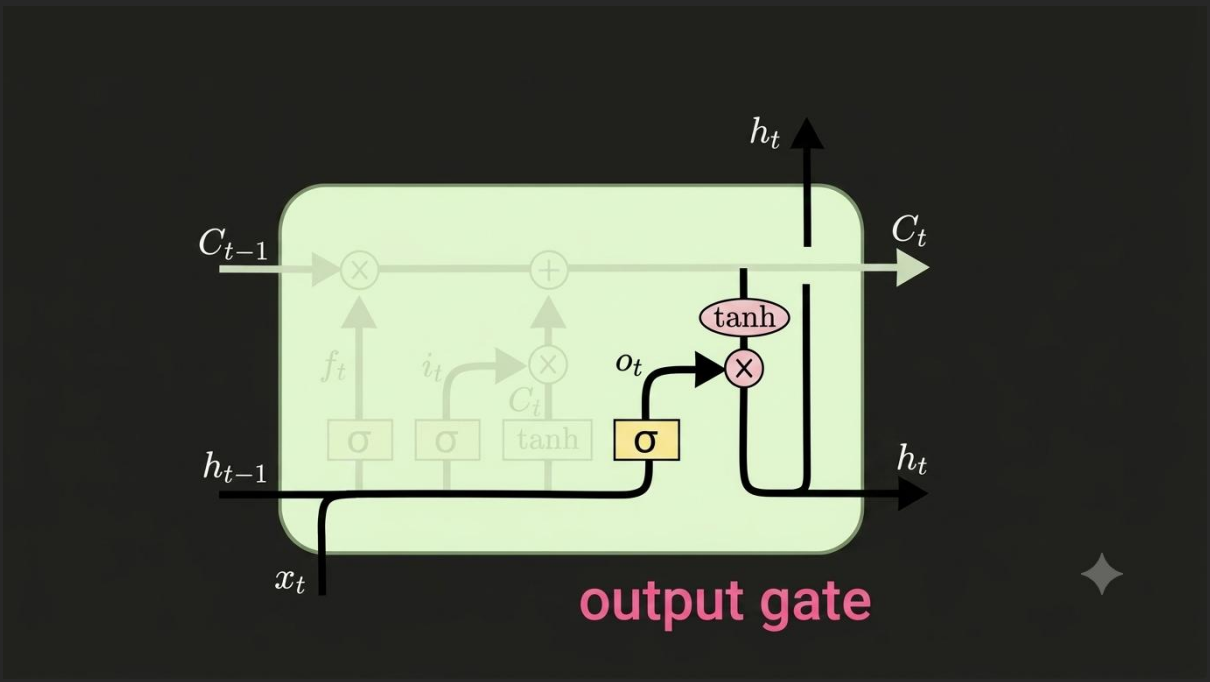
Ravi was an exceptional player, known for his countless centuries and mastery over bowlers. But then came Sunny, who not only surpassed Ravi's records but also rose to become the number one cricketer in the world.




LSTM: Forget Gate (f_t)
h_t
C_{t-1}
C_t
f_t
i_t
q_t
C_t
tanh
σ
σ
tanh
σ
h_{t-1}
h_t
x_t
Neural Network Layer
Pointwise Operation
Vector Transfer
Copy


h_t
C_{t-1}
C_t
tanh
f_t
$i_t = \sigma(...)$
$\tilde{C}_t = \tanh(...)$
v_t
σ
σ
tanh
σ
h_{t-1}
h_t
CANDIDATE CELL STATE
INPUT GATE
x_t
Input Gate Layer (sigmoid)
Candidate Cell State (tanh)

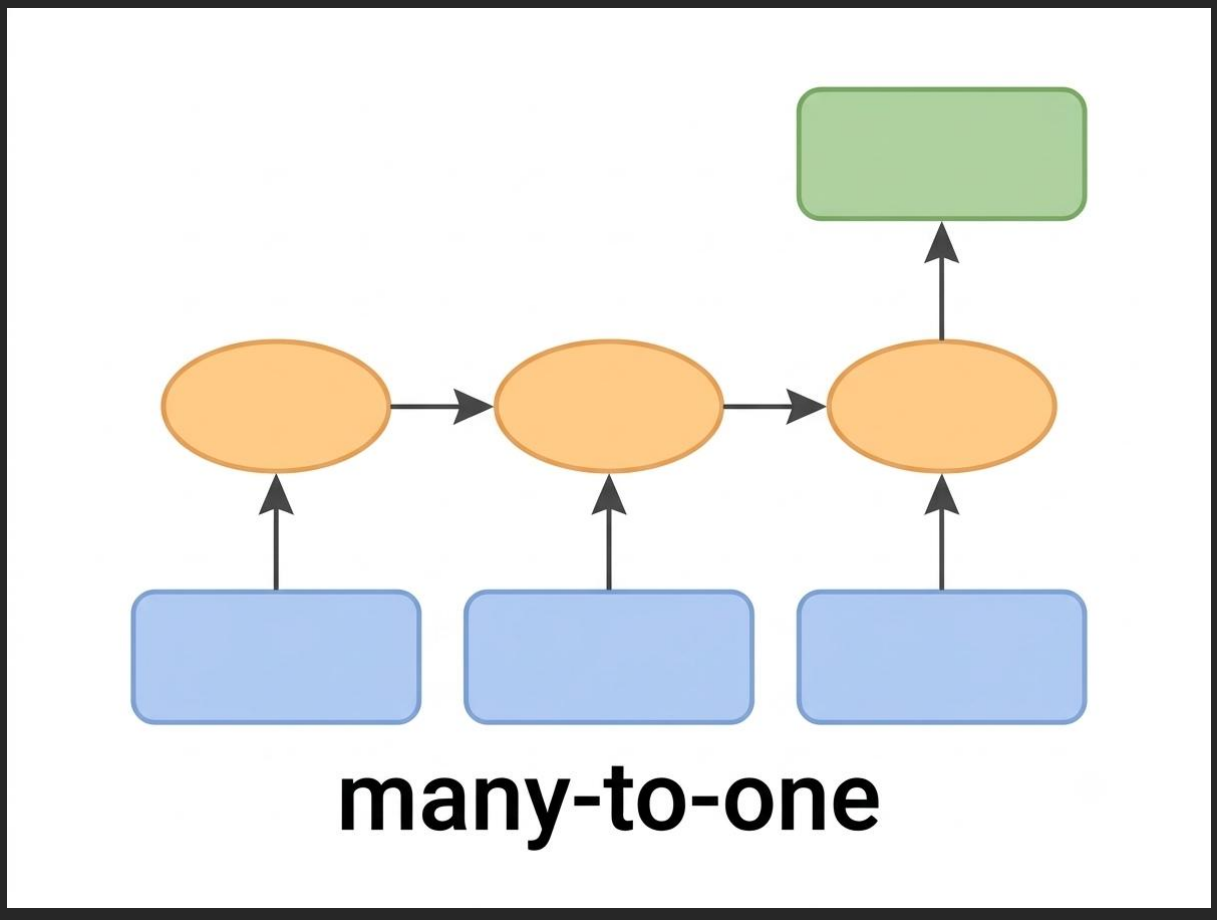


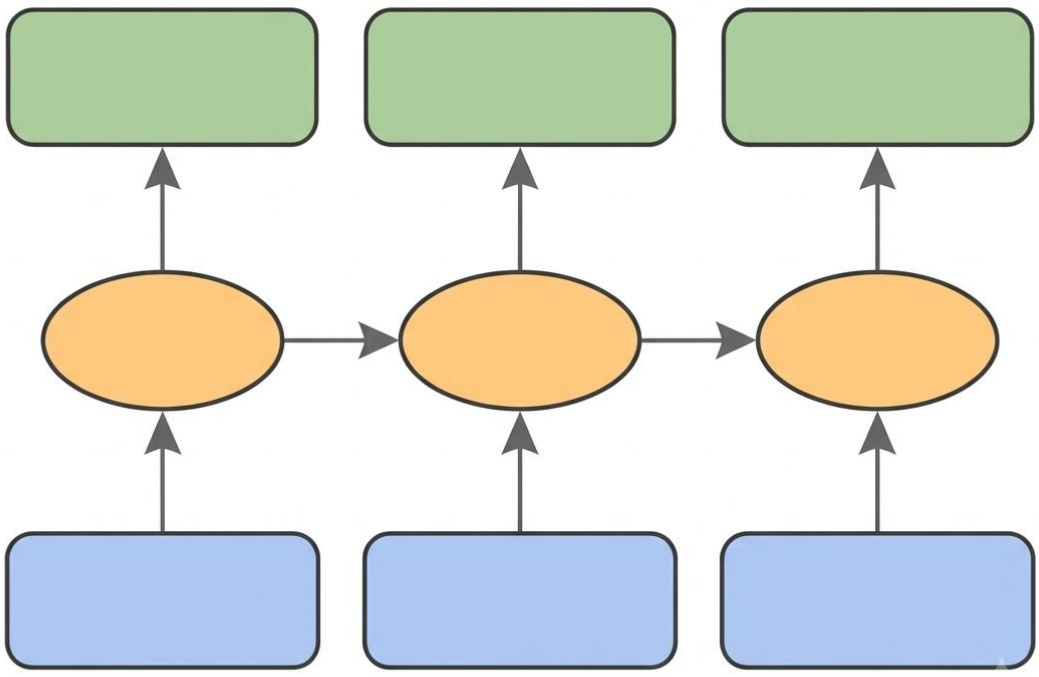


RNN vs LSTM

Feature	RNN	LSTM
Memory	Short-term only	Short + Long-term memory
Gates	No gates	Forget, Input, Output gates

Vanishing Gradient	Yes, very common	Managed somewhat using gates
Training Stability	Not much stable	More stable training
Long dependencies	Fails to capture	Captures better than RNN
Use-cases	Short sequences	Long sequences, context-heavy tasks





many-to-many

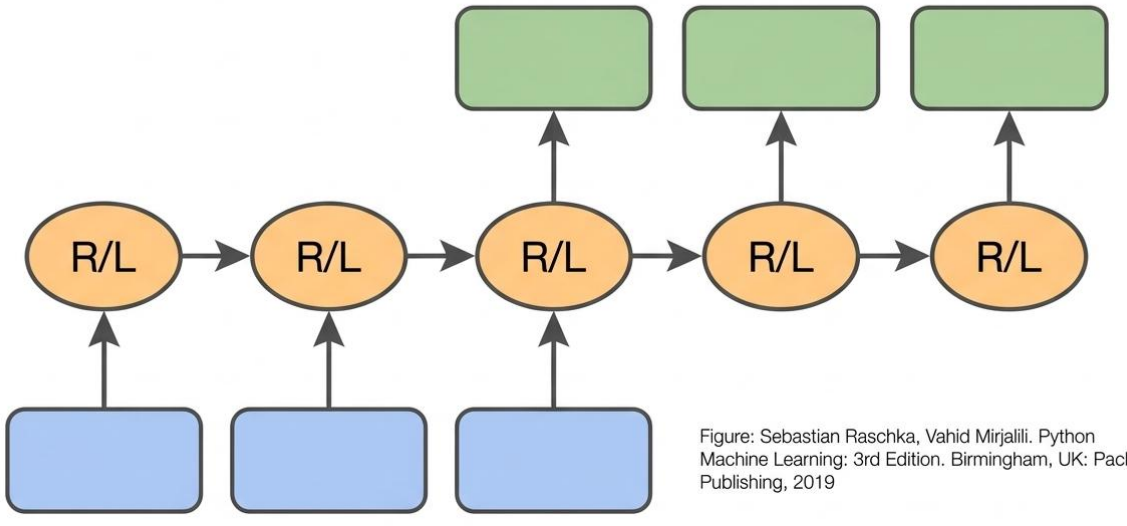
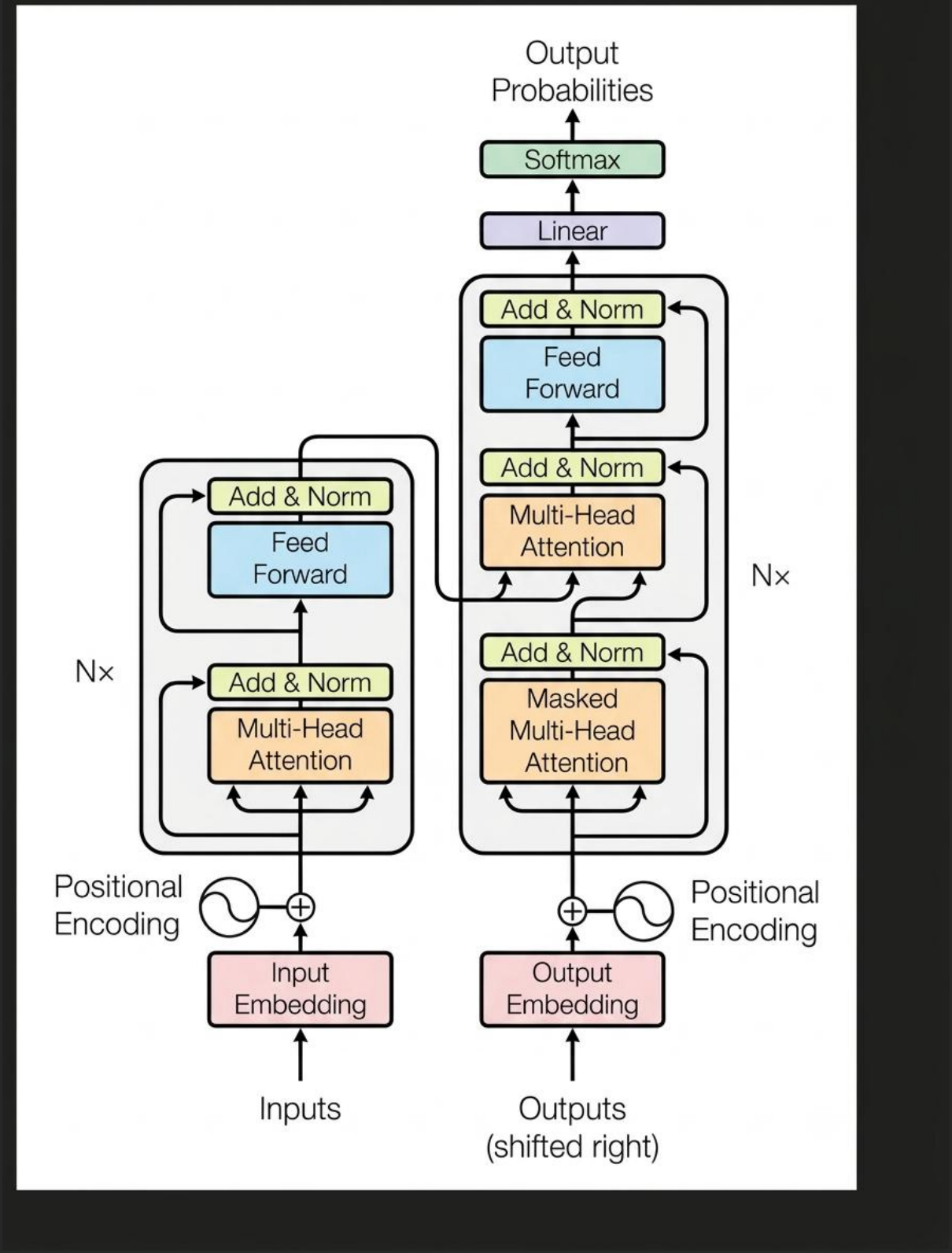


Figure: Sebastian Raschka, Vahid Mirjalili. Python Machine Learning: 3rd Edition. Birmingham, UK: Packt Publishing, 2019

many-to-many



What is the Transformer?

The Transformer is a deep learning architecture introduced by Google in 2017.

It completely replaced RNNs and LSTMs for most NLP tasks (like translation, summarization, Q&A).

It was designed to:

- Handle sequences without using recurrence (RNN)
- Work faster and in parallel
- Capture long-range dependencies using attention

The key idea?

"Let the model focus on the most relevant parts of the input" using something called Self-Attention.

Transformer Encoder — Step-by-Step Process

Summary Flow

- Input: I love mango → Encoder
- Target: मुझे आम पसंद है → Decoder

Training me: Decoder ko sikhaya jata hai har next token predict karna.

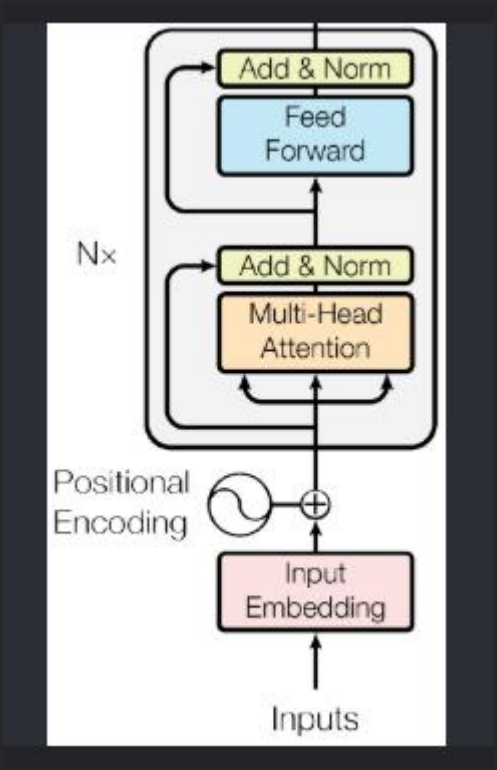
Inference me: Har step ka output next step ka input banta hai.

1. Pass the input sequence as token IDs

Raw text (I love Mango) is first tokenized and converted into a sequence of token IDs (numerical representation of words).

Input: "I love mango"

Tokenized (IDs): [101, 2001, 3567]



2. Token Embedding (Word Embedding)

Each token ID is converted into a dense vector (embedding) using a trainable embedding matrix. This gives semantic meaning to each token.

Word	Token ID	Embedding (dim = 768 or 512 etc.)
I	101	[0.24, -0.15, ..., 0.98]
love	2001	[0.34, 0.02, ..., -0.87]

mango	3567	[0.12, 0.41, ..., 0.33]

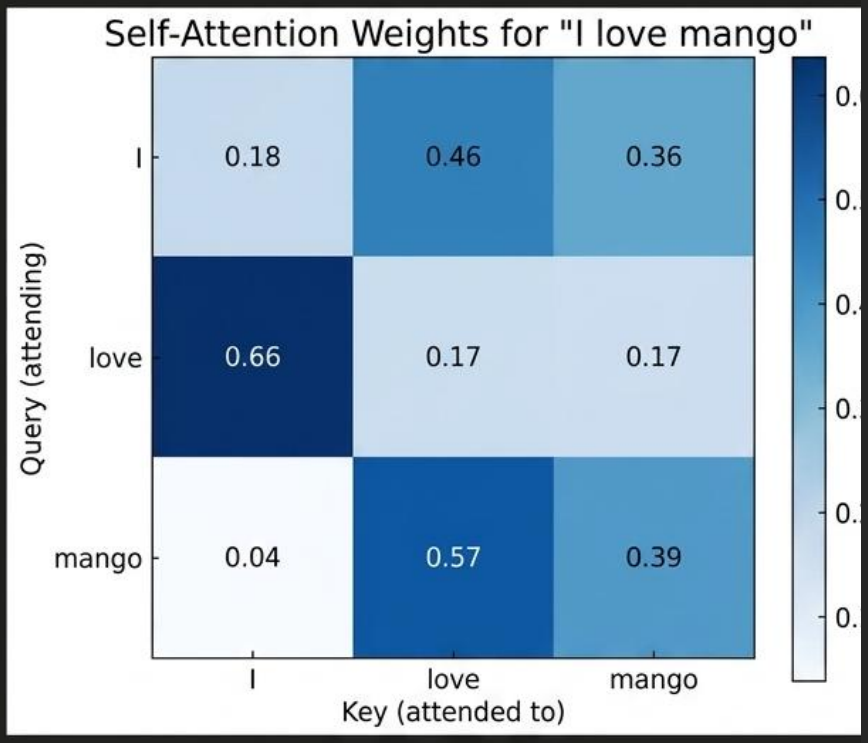
So now we have an input matrix of shape:

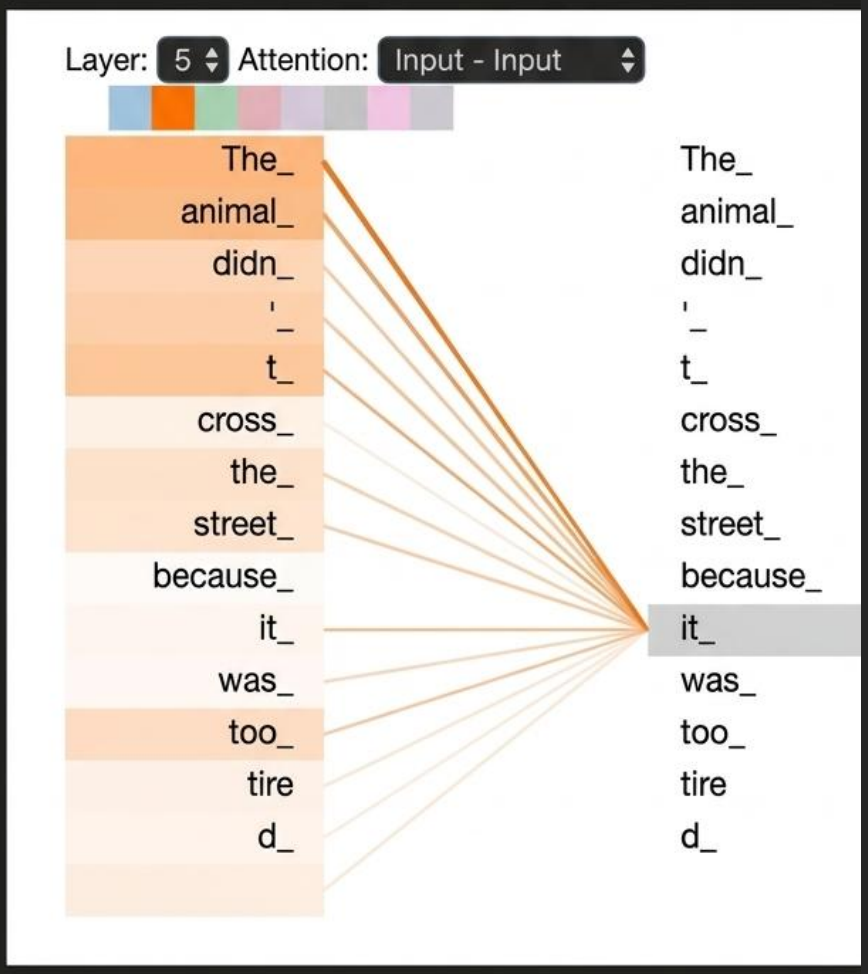
(sequence_length=3, embedding_dim=512)

3. Add Positional Encoding

Since Transformers have no recurrence or convolution, positional encodings are added to the token embeddings to inject information about word order.

Position	Positional Encoding	Final Input
0 ("I")	[0.0, 1.0, ...]	Embed + Pos
1 ("love")	[0.84, 0.52, ...]	Embed + Pos
2 ("mango")	[0.91, 0.23, ...]	Embed + Pos


Self-Attention Weights for "I love mango"
I
love
mango
Query (attending)
0.18
0.46
0.36
0.66
0.17
0.17
0.04
0.57
0.39
I
love
mango
Key (attended to)


Layer: 5
Attention: Input - Input
The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_
The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_

4. Self-Attention (Multi-Headed Self-Attention)

Each token attends to all other tokens in the sequence to learn contextual relationships. Multiple attention heads allow the model to capture different types of dependencies in parallel.

5. Add (Residual Connection) and Layer Normalization

The output of the self-attention layer is added to the original input embedding (residual connection), and the result is normalized using layer normalization.

- Helps gradient flow
- Stabilizes learning
- Speeds up training

6. Feed Forward Network (FFN)

A position-wise fully connected feed-forward neural network is applied to each token independently. Typically, it's a two-layer MLP with a ReLU or GELU activation.

7. Again Add (Residual Connection) and Layer Normalization

A residual connection is applied again (input + FFN output), followed by another layer normalization.

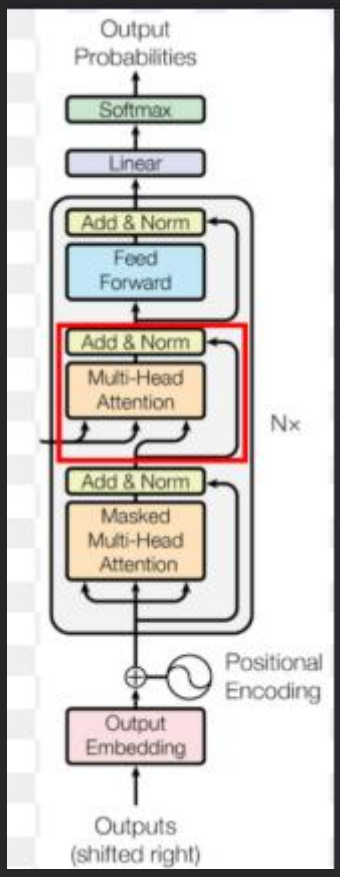
8. Repeat for N Encoder Blocks

N identical encoder layers. In the original research paper, N = 6.

9. All Encoder Blocks are Identical

Each encoder block follows the exact same structure and configuration. Parameters are different (learned separately), but the internal operations are the same in every block.

Transformer Decoder — Step-by-Step Process



1. Shifted Output Sequence

During training, we provide the model with the "ground truth" — but shifted by 1 token.

Ground Truth:

मुझे आम पसंद है

Shifted Input to Decoder:

<s> मुझे आम पसंद

<s> = Start of sentence token

Reason: To teach the model how to predict the next word based on the previous one (teacher forcing).

2. Output Sequence Embedding

<s>, मुझे, आम, ... are converted into vector embeddings.

Each word is represented in vector form (meaning embedding).

Example:

मुझे → [0.12, -0.07, ..., 0.54] (512-dim)

3. Positional Encoding

Since Transformers don't have recurrence (no RNN-style loop), we add position information.

Word Embedding + Positional Info → Final vector

4. Masked Multi-Head Attention

This step ensures that:

- While predicting "मुझे", it can only see <s>
- While predicting "आम", it can see <s> + "मुझे"

- While predicting "पसंद", it can see <s> + "मुझे" + "आम"

The model must not see future words!

5. Add (Residual Connection) & Layer Normalization

The attention output is added to the original input vector (residual connection).

Then layer norm is applied — for stable training.

Output = input + Transformation + Normalize

6. Cross Attention (Multi-head Attention with Encoder Output)

This is where the core translation logic happens — the decoder now attends to the encoder's output.

Example:

- Decoder's "मुझे" → input's "I"
- Decoder's "आम" → input's "mango"
- Decoder's "पसंद" → input's "love"

The model learns mapping between English and Hindi.

7. Add + Norm Again

The cross-attention output is also normalized and passed with residual connection.

8. Feed Forward Neural Network

Each token's vector passes through a small MLP (2-layer neural network):

Linear → ReLU → Linear

9. Residual + Norm Again

Same residual + normalization is applied after the FFN.

10. Linear Layer

Each token's output is converted into a logits vector (based on vocabulary size).

Example:

Logits for "मुझे":

[..., "पसंद": -1.2, "आम": 4.6, "मुझे": +3.5, ...]

11. Softmax Activation

Logits are converted into probabilities:

- "मुझे": 0.80
- "आम": 0.10
- "पसंद": 0.05

The word with the highest probability is selected — here it is "मुझे".

12. Final Output

The model predicts:

- Step 1 → <s> → मुझे
- Step 2 → <s> मुझे → आम
- Step 3 → <s> मुझे आम → पसंद
- Step 4 → <s> मुझे आम पसंद → है

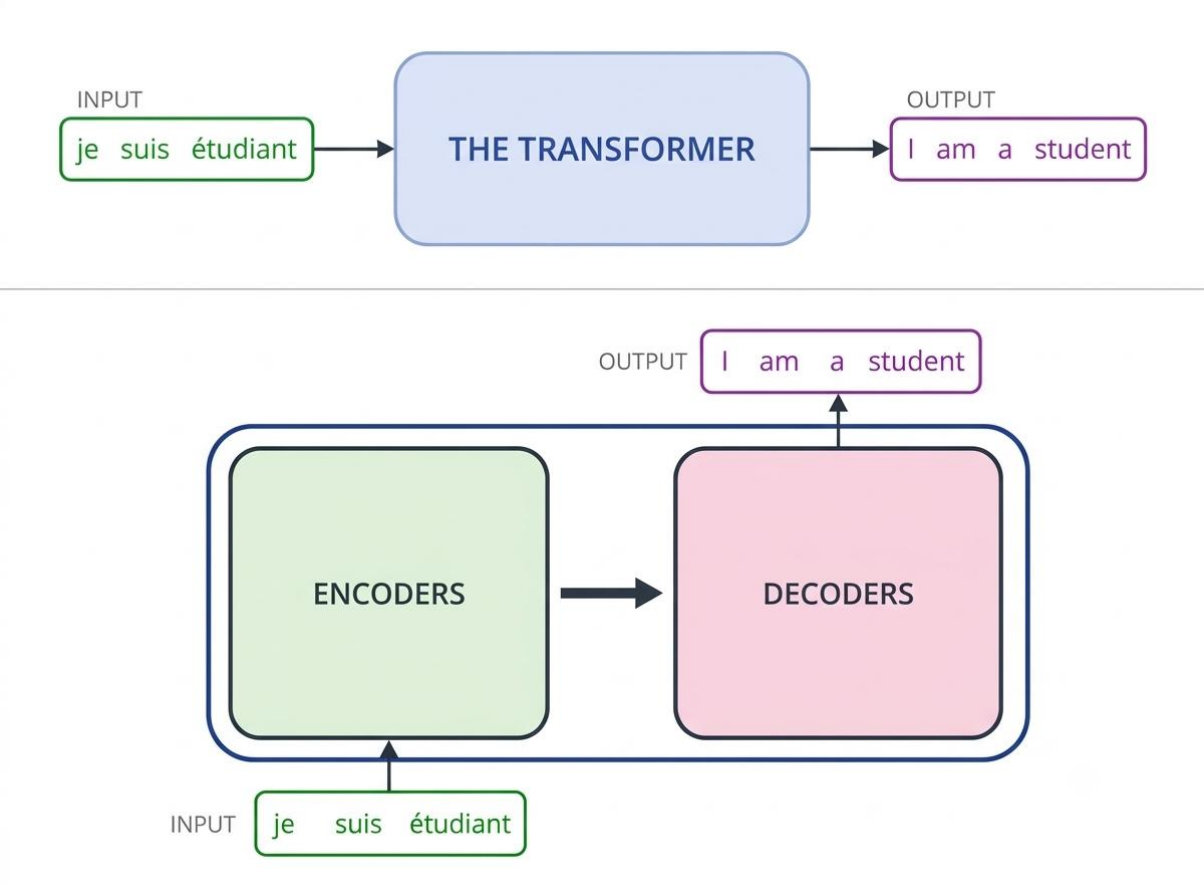
Output: मुझे आम पसंद है

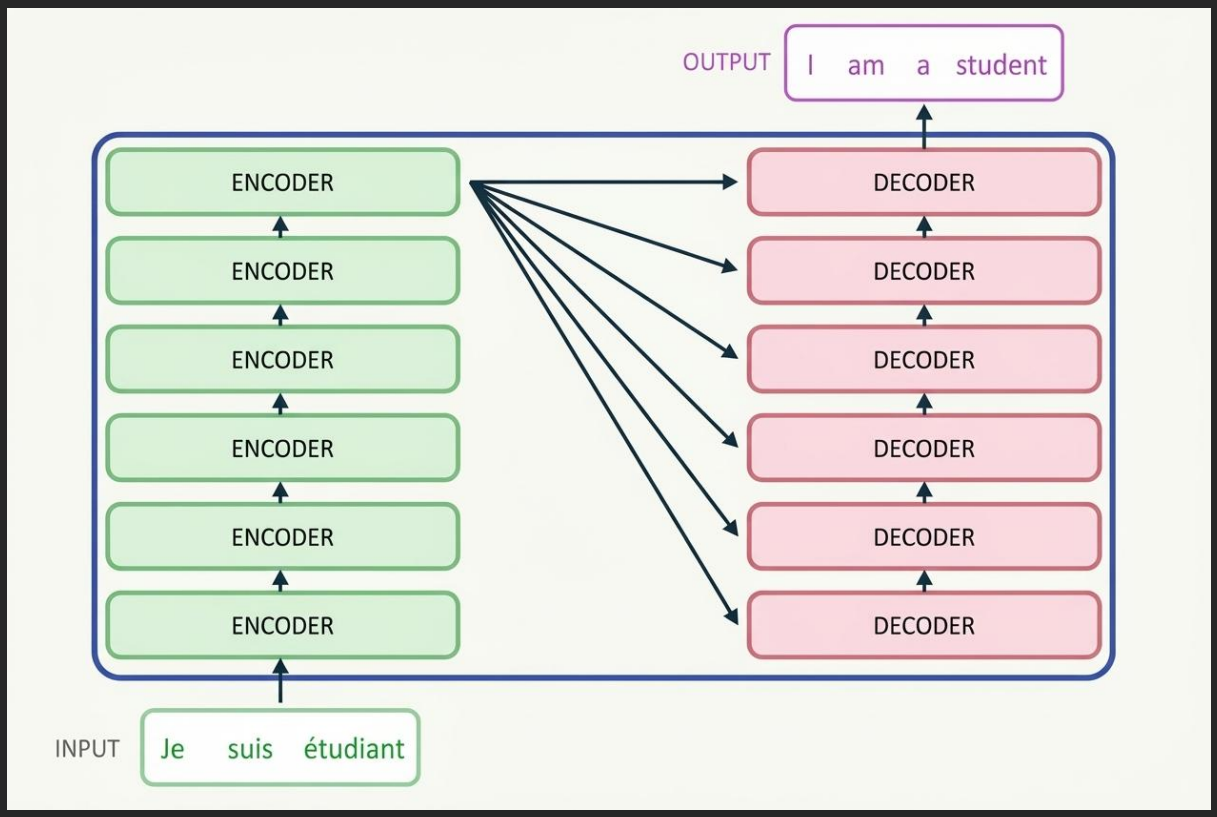
13. Multiple Decoder Blocks

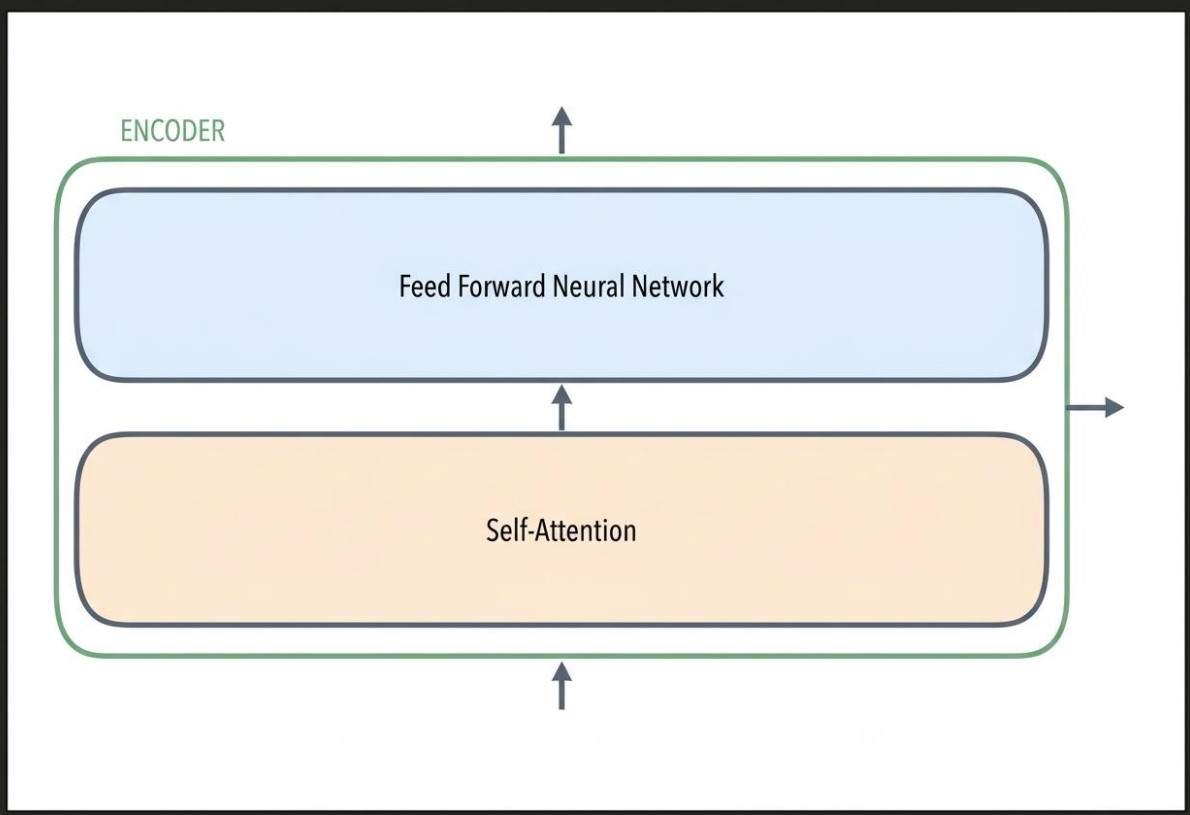
This entire process is repeated 6 times (stacked decoder blocks).

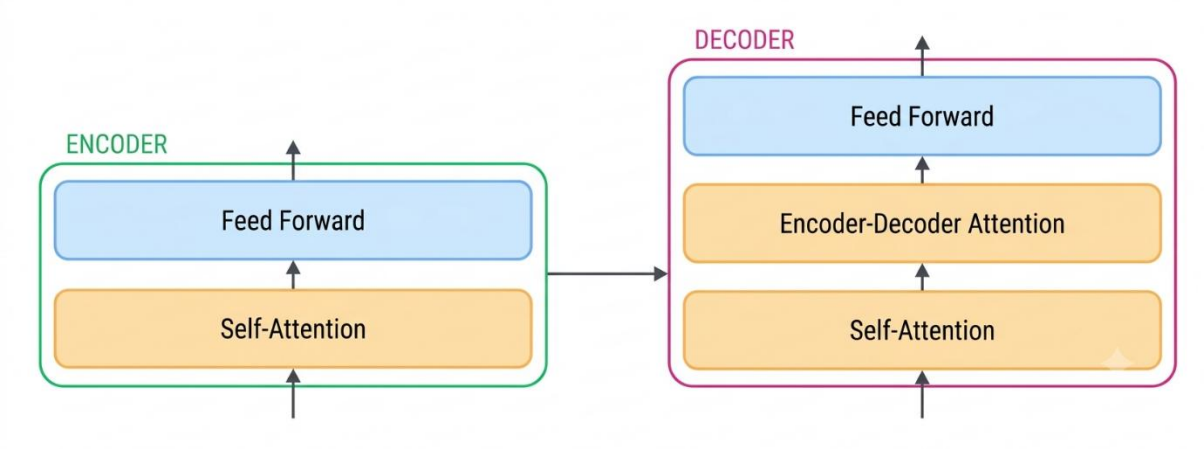
Each block further refines the prediction.

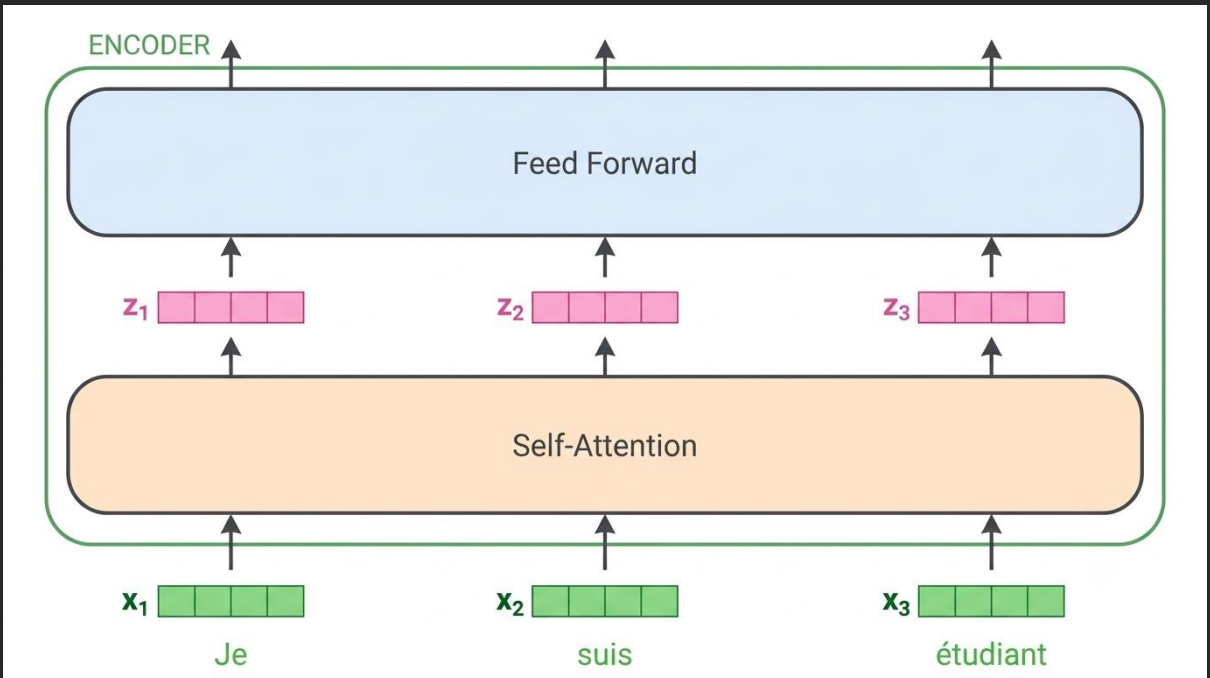
The final decoder block produces the final output.

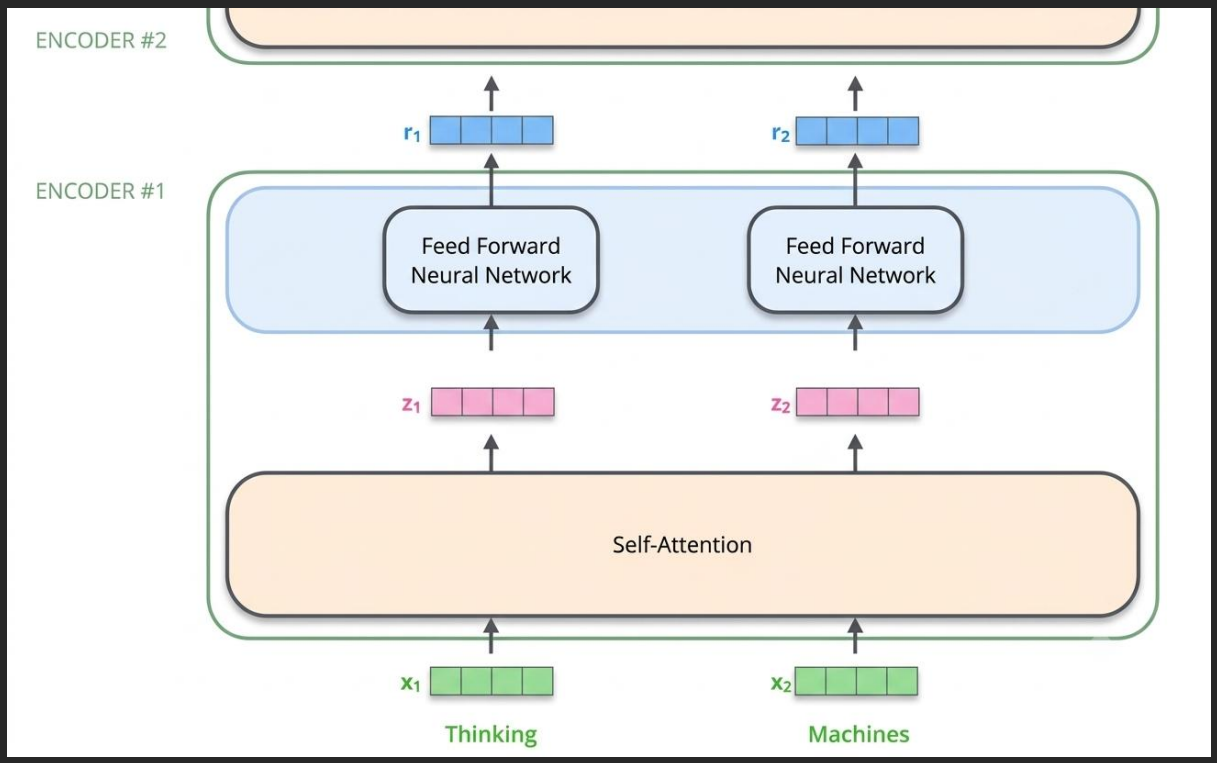


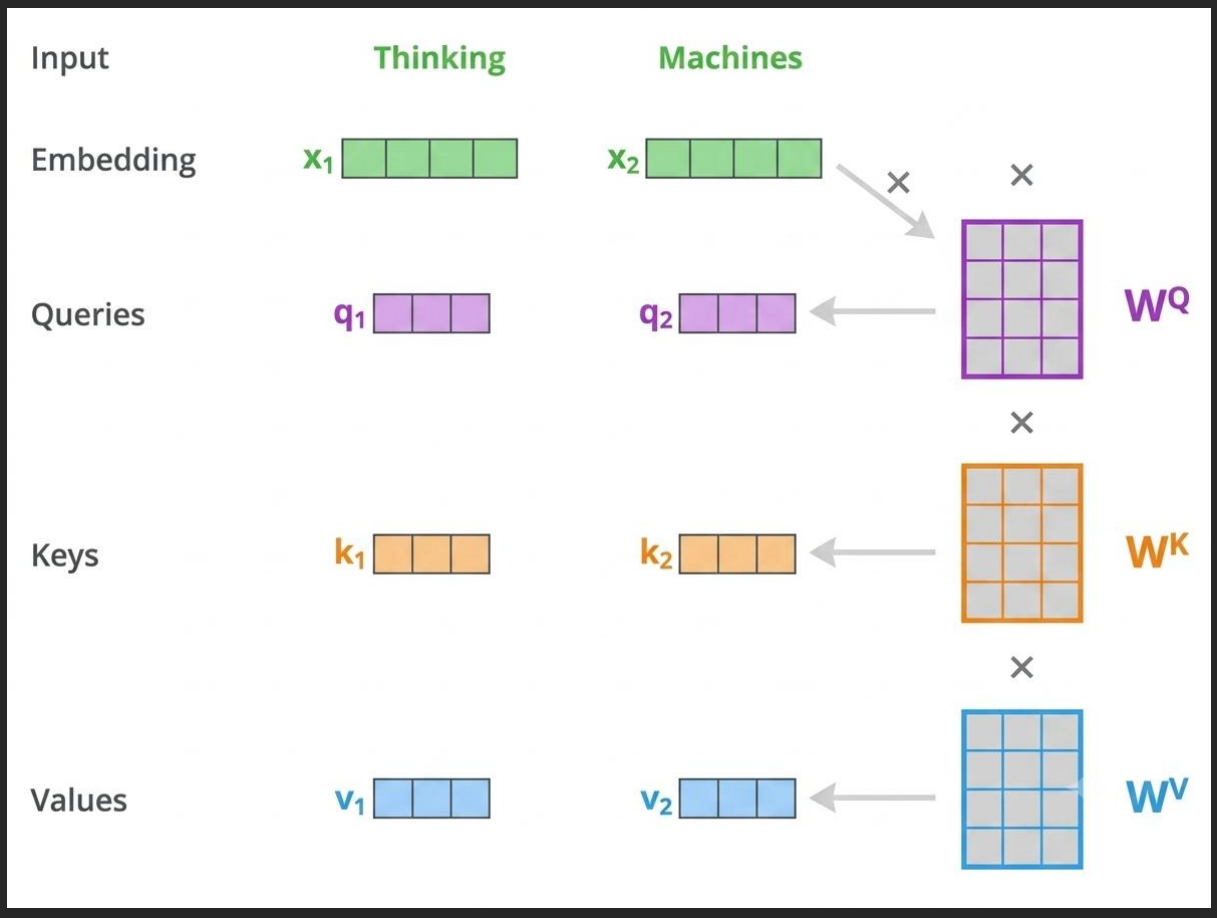

OUTPUT
I am a student
ENCODER
ENCODER
ENCODER
ENCODER
ENCODER
ENCODER
DECODER
DECODER
DECODER
DECODER
DECODER
DECODER
INPUT
Je suis étudiant

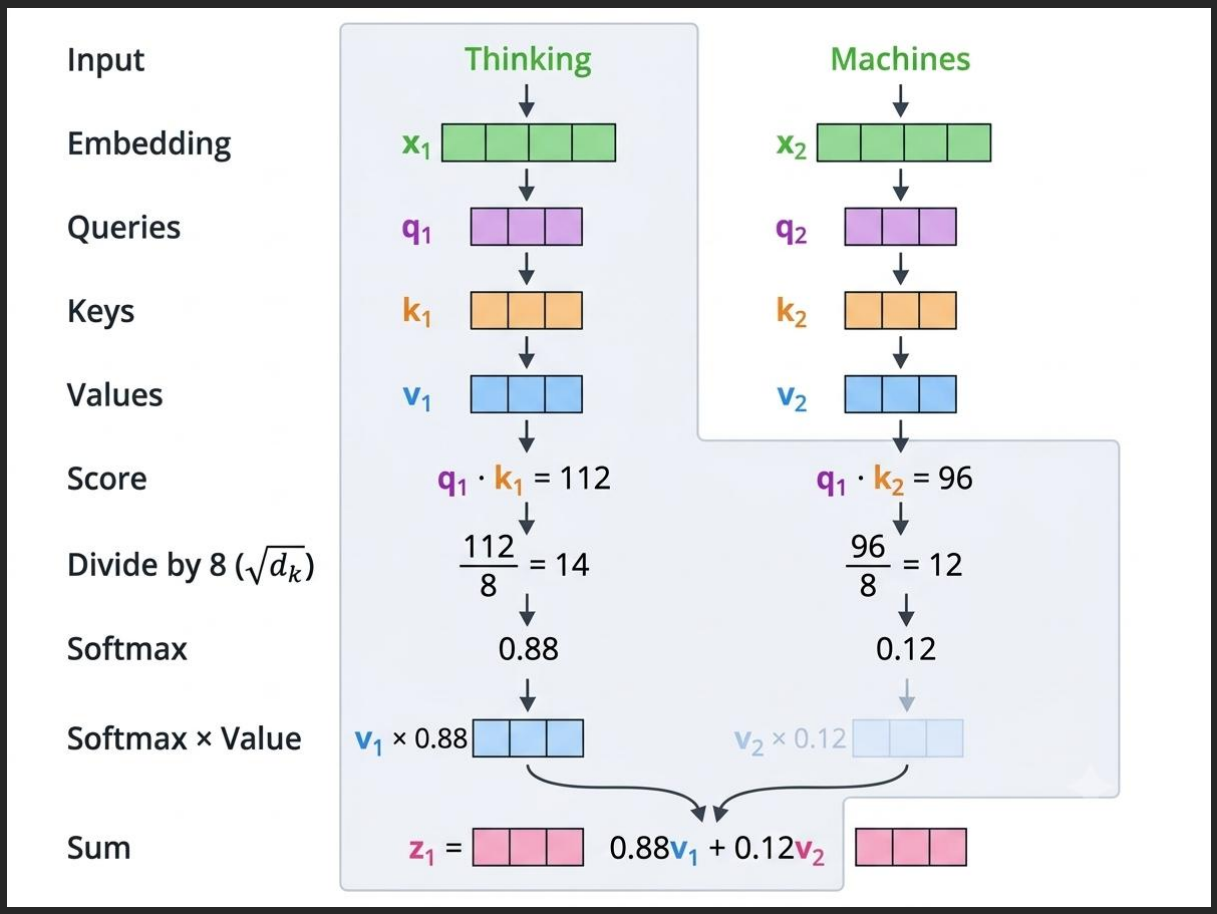

ENCODER
Feed Forward Neural Network
Self-Attention

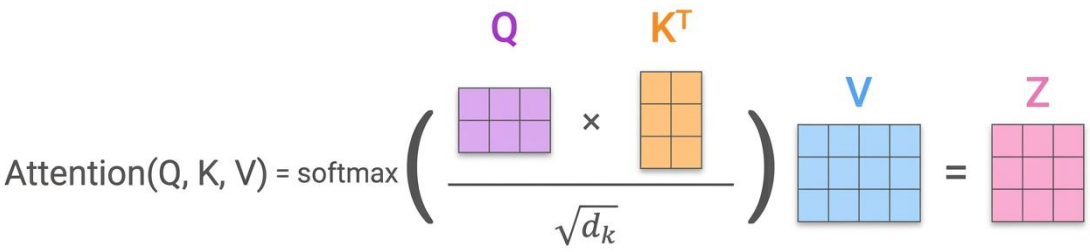

DECODER
Feed Forward
Encoder-Decoder Attention
Self-Attention
ENCODER
Feed Forward
Self-Attention


ENCODER
Feed Forward
z_1
z_2
z_3
Self-Attention
x_1
x_2
x_3
Je
suis
étudiant


ENCODER #2
r1
r2
ENCODER #1
Feed Forward Neural Network
Feed Forward Neural Network
z1
z2
Self-Attention
x1
x2
Thinking
Machines


Input
Thinking
Machines
Embedding
x1
x2
Queries
q1
q2
WQ
Keys
k1
k2
WK
Values
v1
v2
WV





LSTM v/s Transformer

Feature	RNN	LSTM	Transformer

Architecture	Recurrent	Recurrent (gated)	Attention-based (non-recurrent)
Handles Long Dependencies	No (forgets quickly)	Better than RNN (with memory cells)	Excellent (global context)
Training Parallelism	Sequential (slow)	Sequential (still slow)	Full parallelism (fast)
Vanishing Gradient	Common	Less common (gates help)	Not a problem
Memory / Context Window	Small	Medium	Large / Global
Suitable for	Small text, time-series	Medium text, sequences	Long text, translation, summarization, generation
Position Tracking	Built-in (via sequence)	Built-in	Uses positional encoding

Attention Support	No	No	Native self-attention
Inference Speed	Slow	Still slow	Fast with GPU
Pretraining Available	Rare or no	Rare or no	Yes (BERT, GPT, T5, Llama, Mistral etc.)

Why is this RNN/LSTM model not the right fit for fine-tuning?

In LSTM models fine-tuning was possible partially — but it doesn't scale well.

As a result, a robust pretrained model ecosystem never emerged around LSTMs, unlike Transformers in NLP or CNNs in computer vision.

Reason 1: No Parallelism = Slow Training & Inference

- RNN/LSTM process input token by token (sequentially) → No GPU-level parallelism
- Transformers process entire sequence in parallel using matrix multiplication → Leverages GPU fully
- Fine-tuning needs fast iteration over large datasets — RNNs just can't keep up

LSTMs Can't Scale to Massive Pretraining Datasets

- LSTMs operate sequentially — processing one token at a time — and therefore cannot be parallelized efficiently on GPUs.
- Now imagine trying to train an LSTM on a billion tokens — it would be painfully slow.
- In contrast, Transformers can process entire sequences in parallel, making them blazingly fast on GPUs.
- As a result, the industry never seriously invested in training LSTMs on web-scale corpora, and no major pretrained LSTM checkpoints were ever created.

Reason 2: Poor Long-Range Dependency Handling

- RNNs forget past tokens quickly due to vanishing gradients.
- LSTMs are slightly better (via memory cells), but still limited to short-to-medium contexts.
- Transformers use self-attention to model global context — every word attends to every other word.
- Summarization, document QA, chat history → all need global context, which RNNs simply can't handle.

They Forget — Catastrophic Forgetting + Limited Memory

- LSTM has limited context window.
- Long dependencies? It forgets quickly.
- So even if you pretrained, it wouldn't generalize well to other tasks/domains.
- Transformer: Global self-attention → remembers whole context.

Reason 3: No Universal Architecture Standard like "CNN"

- In computer vision, unified blueprints like ResNet and VGG were established.
- In NLP, LSTM was the wild wild west.
- Every paper had different layers, units, bidirectionality, attention mechanisms, etc.
- No community standard → no community models.

Reason 4: No Pretrained Models / Transfer Learning Support

- Transformer models like BERT, GPT, LLAMA are pretrained on massive corpora.
- You can fine-tune them on small downstream datasets.

- RNN/LSTM models have no large-scale pretrained weights available today.
- You'd have to train from scratch, which is not feasible.
- Fine-tuning works best when the pretrained base is strong — not possible with raw LSTM models today.

Reason 5: Rise of BERT & GPT Destroyed LSTM Hype (2018 onwards)

- BERT (2018) showed that self-supervised pretraining combined with fine-tuning works incredibly well.
- The industry completely shifted to Transformer-based architectures.
- LSTMs stopped being considered altogether (except in time-series and low-resource devices).

Example:

BERT was trained on 3.3 billion words in just 4 days.

LSTM would take weeks to achieve the same task.

Reason 6: No Infrastructure / Community Support

- There are thousands of BERT models available on Hugging Face.

- Pretrained LSTM models? Zero.